LEARNING WHO TO LISTEN TO IN GRAPHS: REPRODUCING GRAPH ATTENTION NETWORKS

Authors: Layla Liu, Casper Liao, Cornell University, CS 4782: Intro to Deep Learning

INTRODUCTION/MOTIVATION

CNNs **revolutionized** vision by exploiting local structure

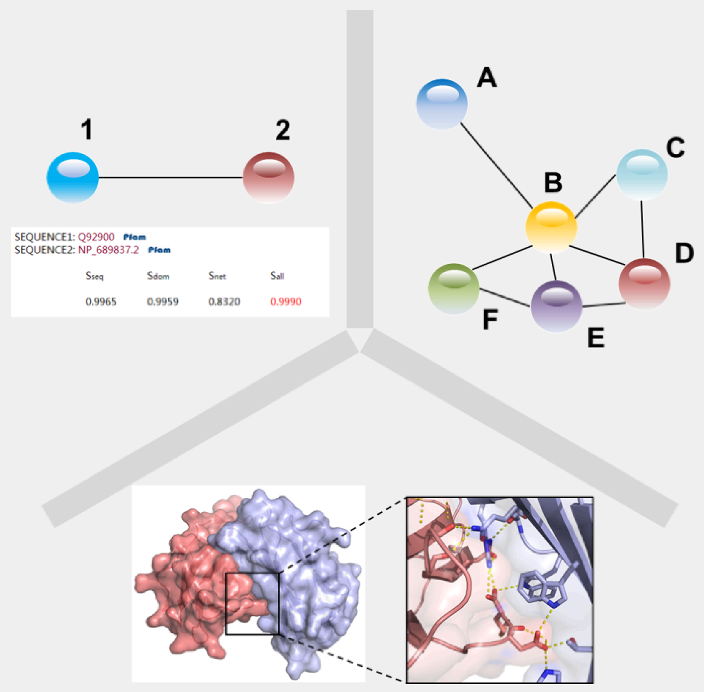
WHAT MAKES CNNS SO POWERFUL

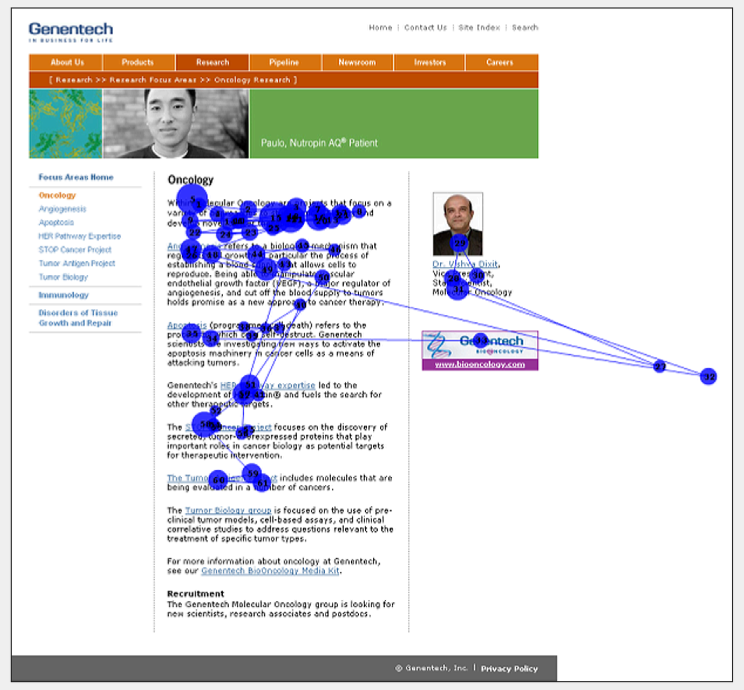
LOCAL RECEPTIVE FIELDS

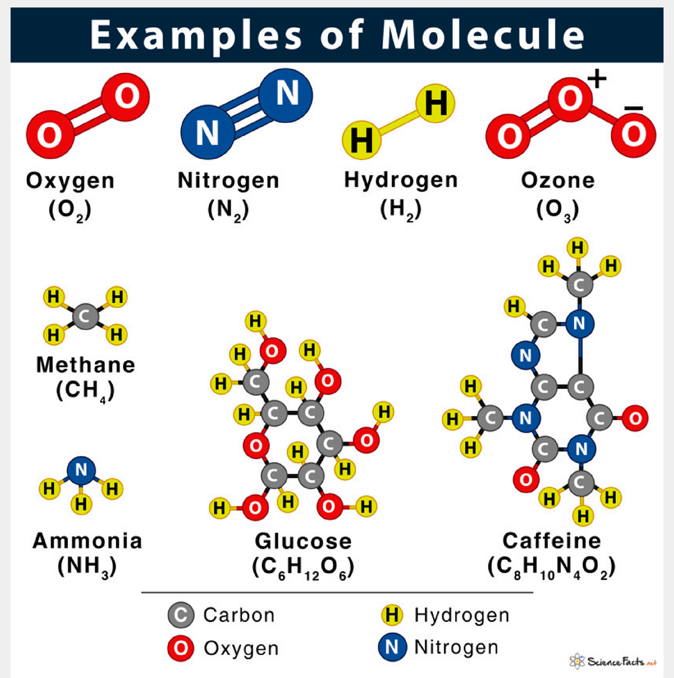
WEIGHT SHARING

TRANSLATION INVARIANCE

GRAPHS ARE EVERYWHERE

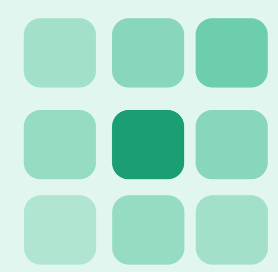






WHY - THE PROBLEM

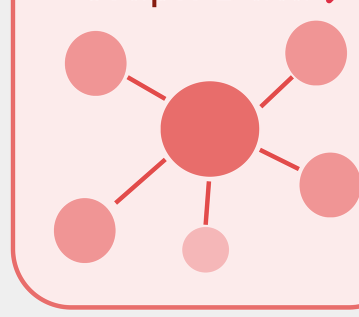
Grid Data ✅



Fixed Structure

FAILS

Graph Data ❌



Irregular, variable

GAT'S CORE INSIGHT

USE GRAPH STRUCTURE AS ATTENTION MASKS

PRIOR APPROACHES - ALL FLAWED

GNNs	Spectral GCN	GraphSAGE
Iterative update	Laplacian basis	Fixed sampling
Slow, not scalable	Can't transfer graphs	Loses neighborhood

GAT solves all three ✅

Efficient — No eigendecompositions
Expressive — Learns which neighbors matter
Inductive — Works on unseen graphs

GOAL

Re-implement Graph Attention Networks in PyTorch and test whether learned node-specific attention improves node classification over uniform aggregation.

REPRODUCE TABLE 2

81.5%	83.0%	82%
GCN baseline	GAT (paper)	GAT (ours)

HOW WE BUILT IT?

STEP 1 — SCORE NEIGHBORS (Eq. 1)

$$e_{ij} = \mathrm{LeakyReLU}(\mathbf{a}^\top [\mathbf{W}\mathbf{h}_i \,\|\, \mathbf{W}\mathbf{h}_j])$$

STEP 2 — NORMALIZE (Eq. 2)

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k \in N(i)} \exp(e_{ik})}$$

STEP 3 — AGGREGATE (Eq. 3)

$$\mathbf{h}'_i = \sigma\left(\sum_{j \in N(i)} \alpha_{ij} \mathbf{W}\mathbf{h}_j\right)$$

MODEL ARCHITECTURE

Input
N x 1433 features

GAT Layer 1
8 heads × 8 dim
ELU activation

GAT Layer 2
1 head → 7 classessoftmax

Prediction

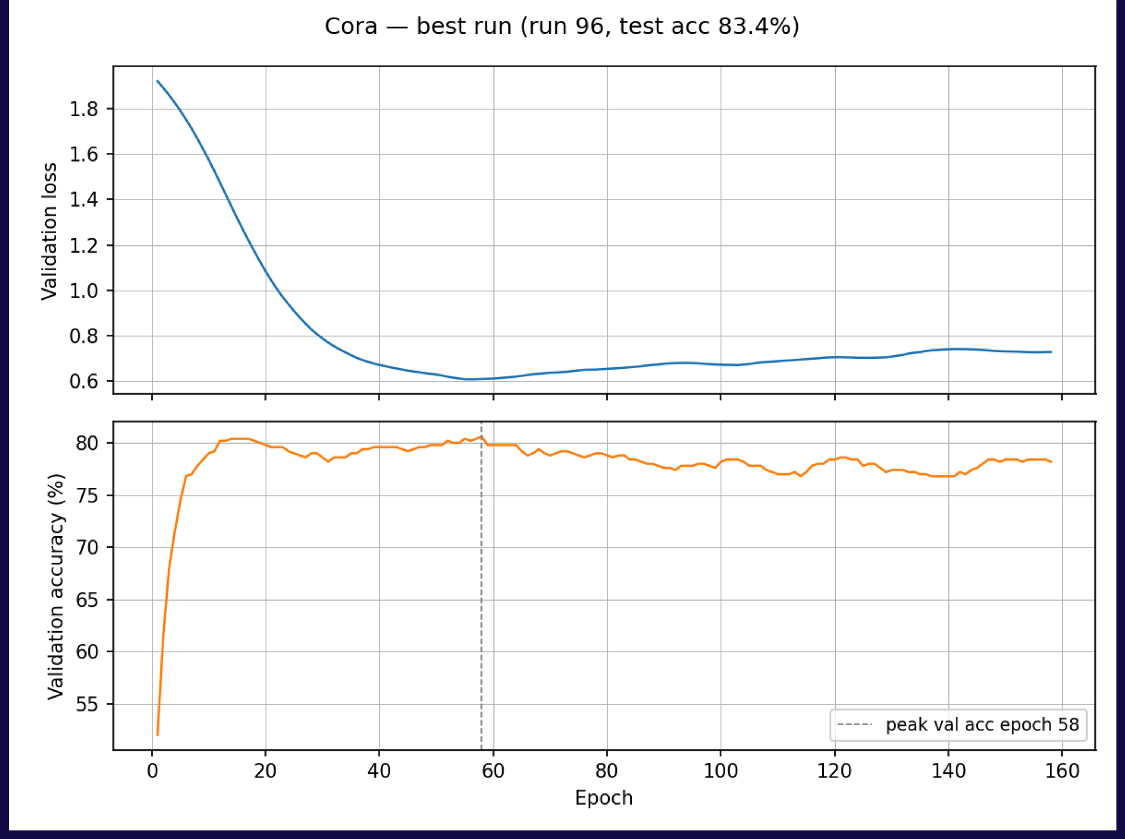
self-loops added so nodes attend to own features

RESULTS

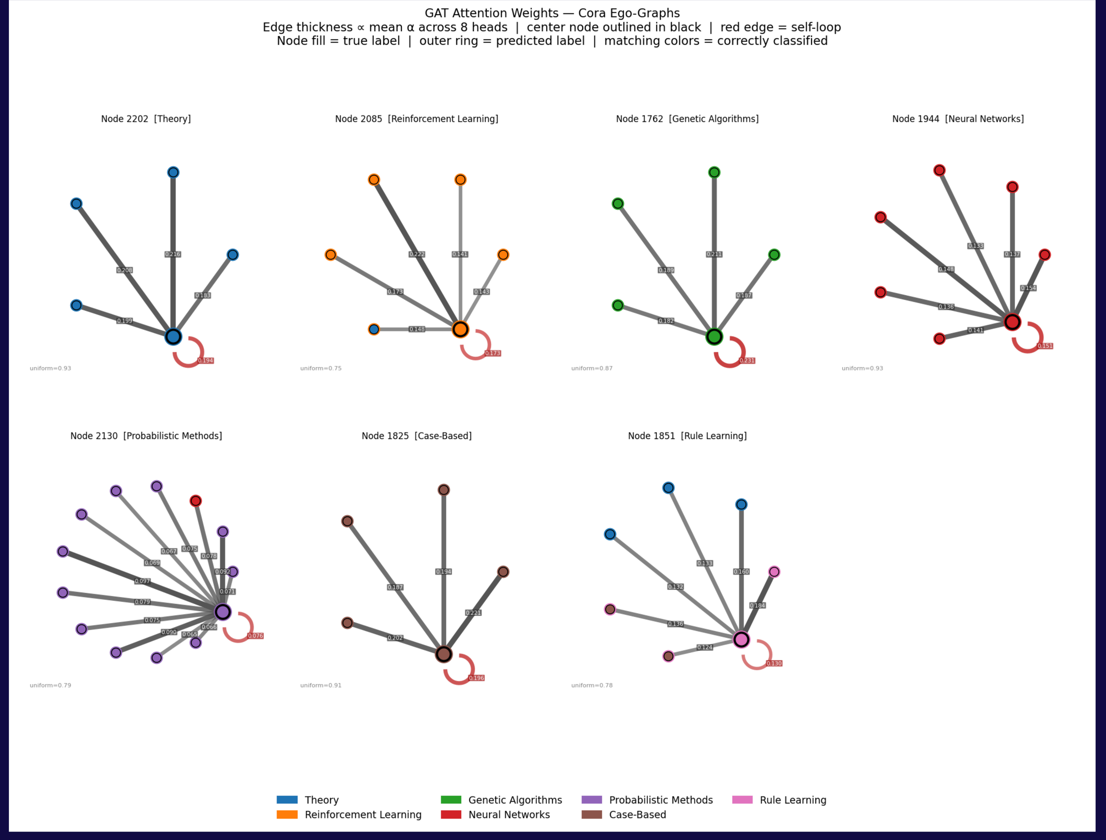
Data Stats: Small accuracy gap

Method	Cora	Citeseer
MLP (no graph structure)	55.10%	46.50%
GCN	81.50%	70.30%
GAT - Paper	83.0 ± 0.7%	72.5 ± 0.7%
GAT - Ours	80.9 ± 1.0%	69.0 ± 1.2%

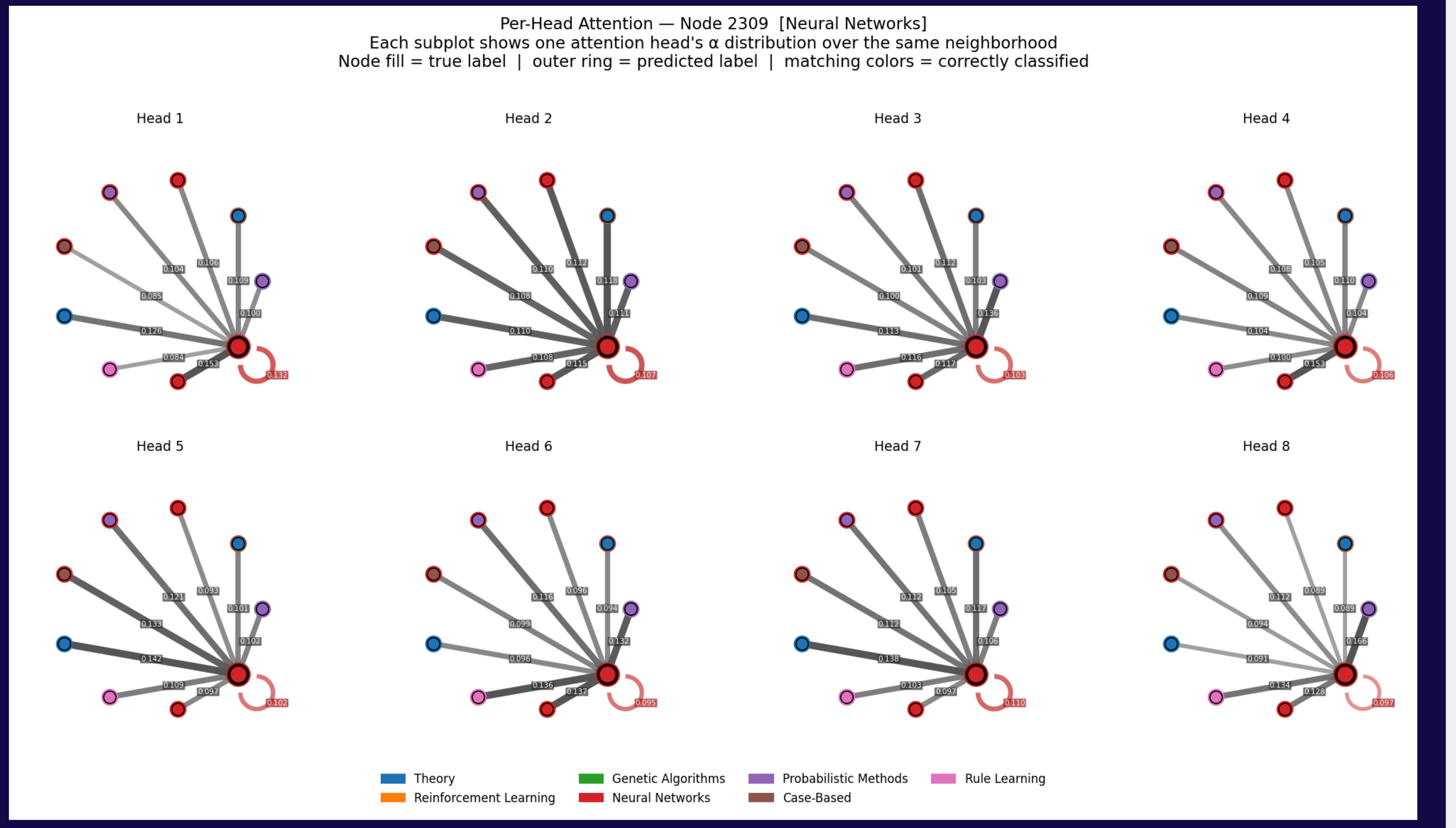
Fast convergence, stay stable.
Early stopping for best model checkpoint.



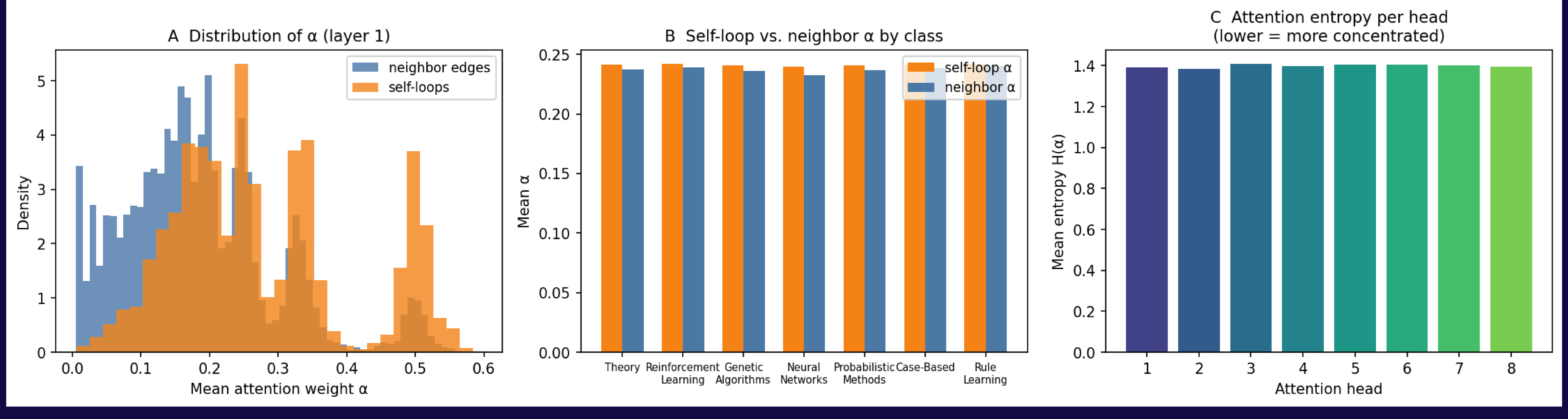
Higher attention weight on same-label neighbor



Concentrated v.s. Uniform weight distribution for different attention heads



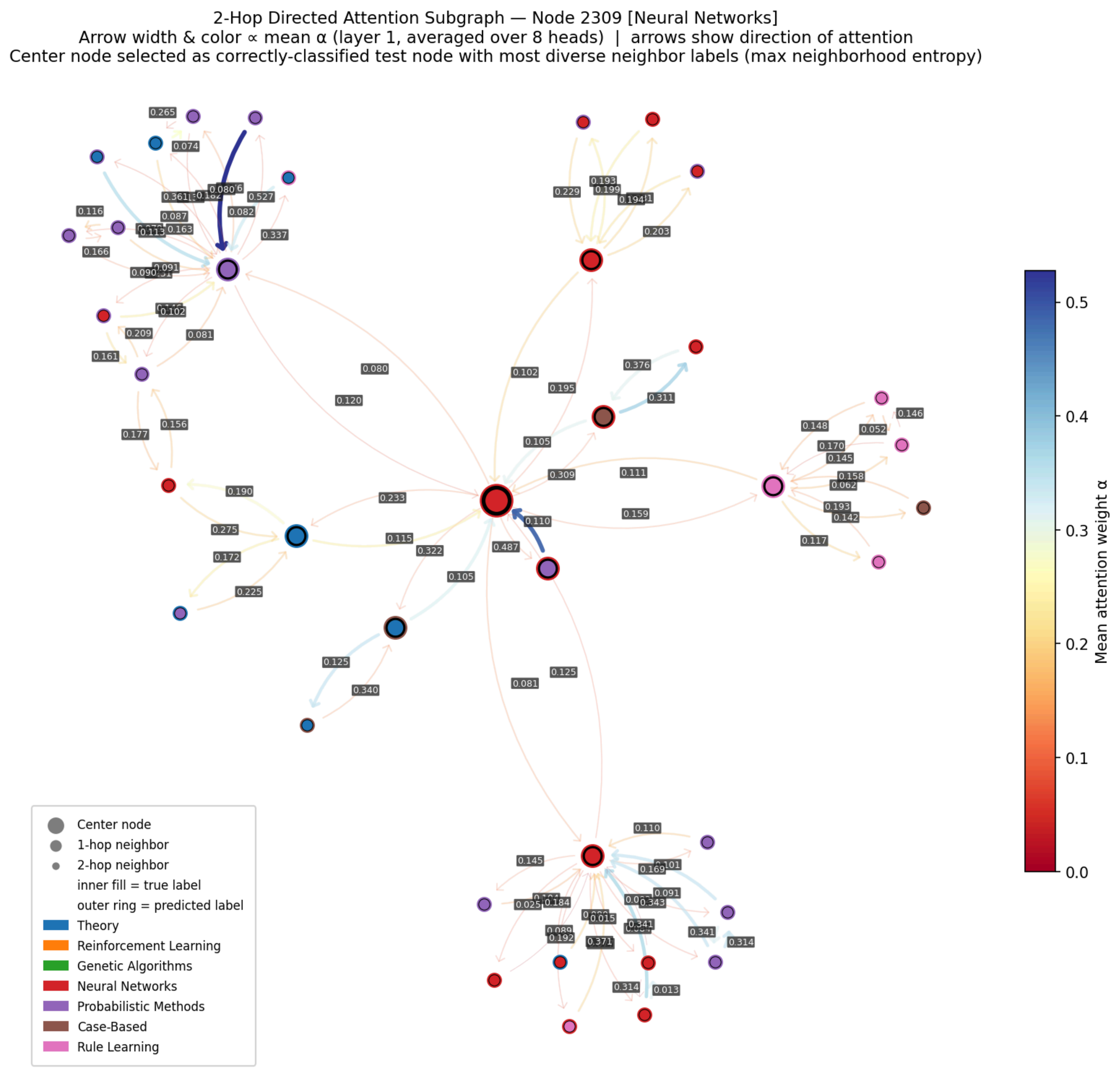
Self loop has more weights. Attention heads have unifrom entropy on average.



CONCLUSION

Learned attention, not just graph structure

Accuracy Gap
- Early stopping criteria
- PyTorch vs TensorFlow initialization differences



Future Works
- heads ablation (K=1/2/4/8)
- layer ablation (test neighbor info smoothness)

REFERENCES: VELIČKOVIĆ 2018, KIPF & WELLING 2017, CORA/CITESEER (SEN ET AL. 2008)